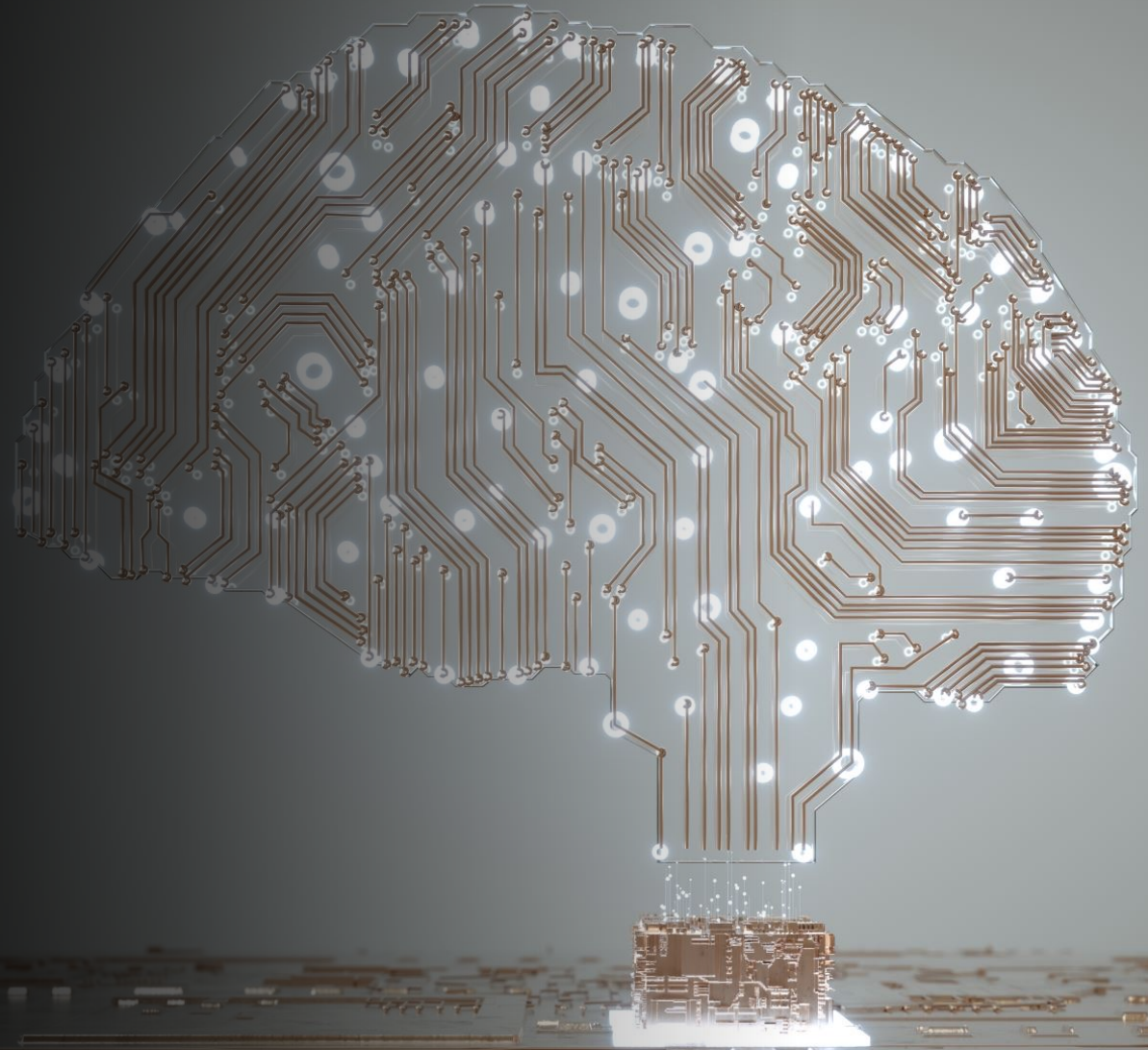


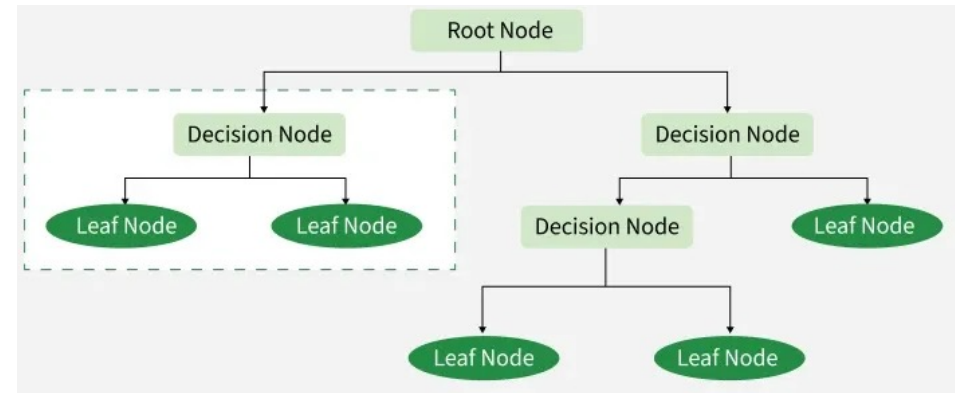
Machine Learning, Decision Tree

Dr. Ahmed Aljarah



Decision tree

- A decision tree is a supervised learning algorithm used for both classification and regression.
- It has a hierarchical structure made up of a root node, internal nodes, branches, and leaf nodes.
- It works like a flowchart, guiding decisions step by step:
 - Root node
 - Internal nodes represent tests on features
 - Branches represent the outcomes of those tests
 - Leaf nodes represent the final prediction or decision
- Decision trees are popular because they are easy to understand, flexible, and require minimal data preprocessing.



Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.

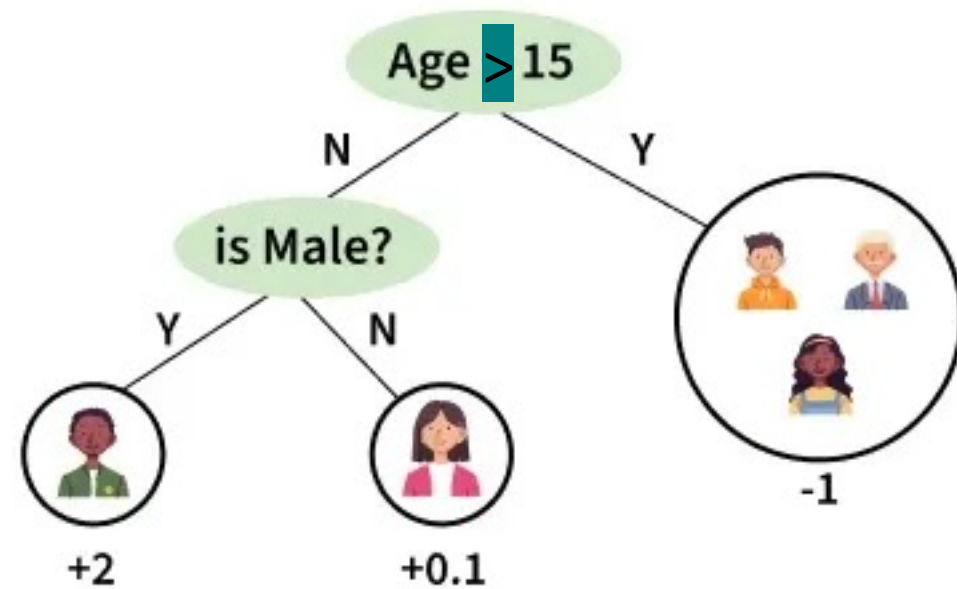
Input: Age, Gender,
Occupation,...



Prediction score in each leaf

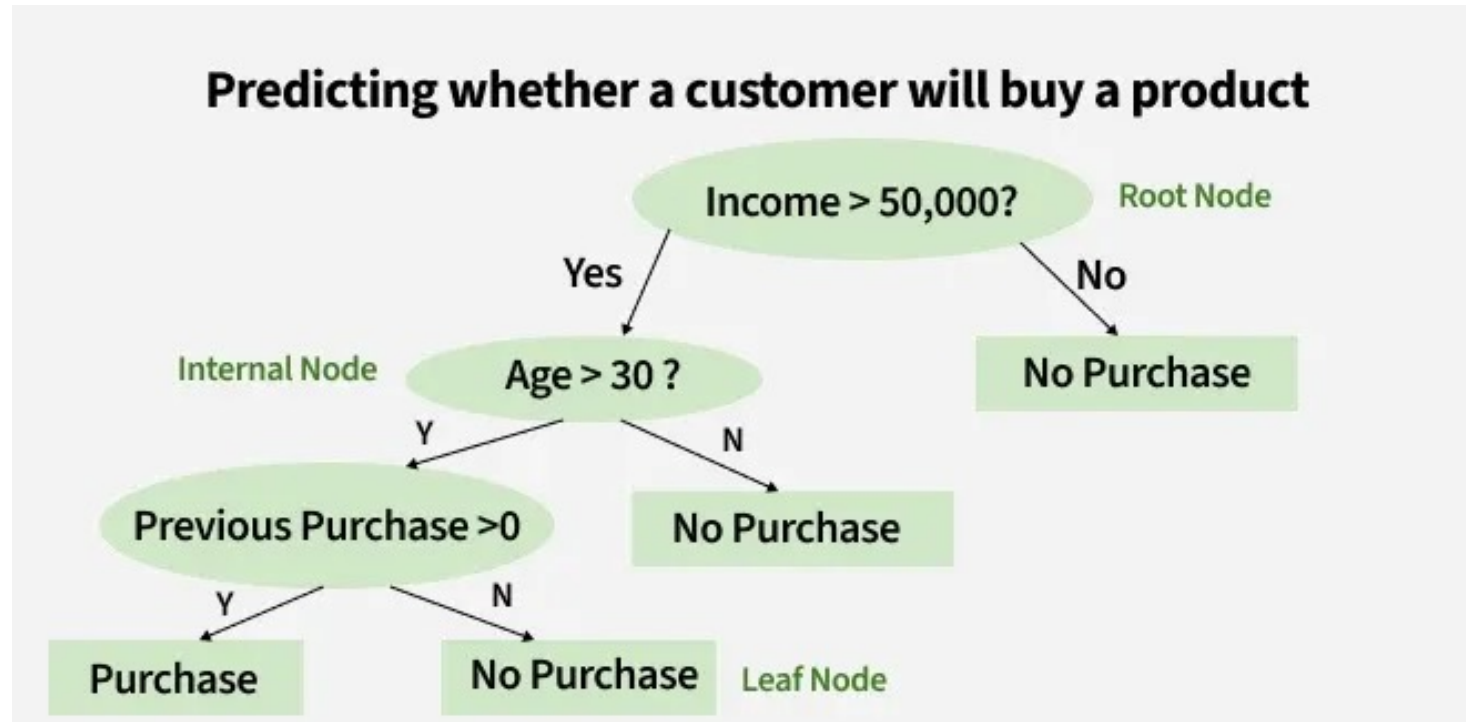


Does the person likes computer games



Data splitting of Decision Tree

- A decision tree splits data using feature values to create groups where items ideally belong to the same class.
- Leaf nodes represent class labels, and internal nodes are decision points based on features.



Explanation

1. Root Node (Income):

First question: *"Is the income greater than \$50,000?"*

1. Yes, move to the next step
2. No, predict "No Purchase"

2. Internal Node (Age):

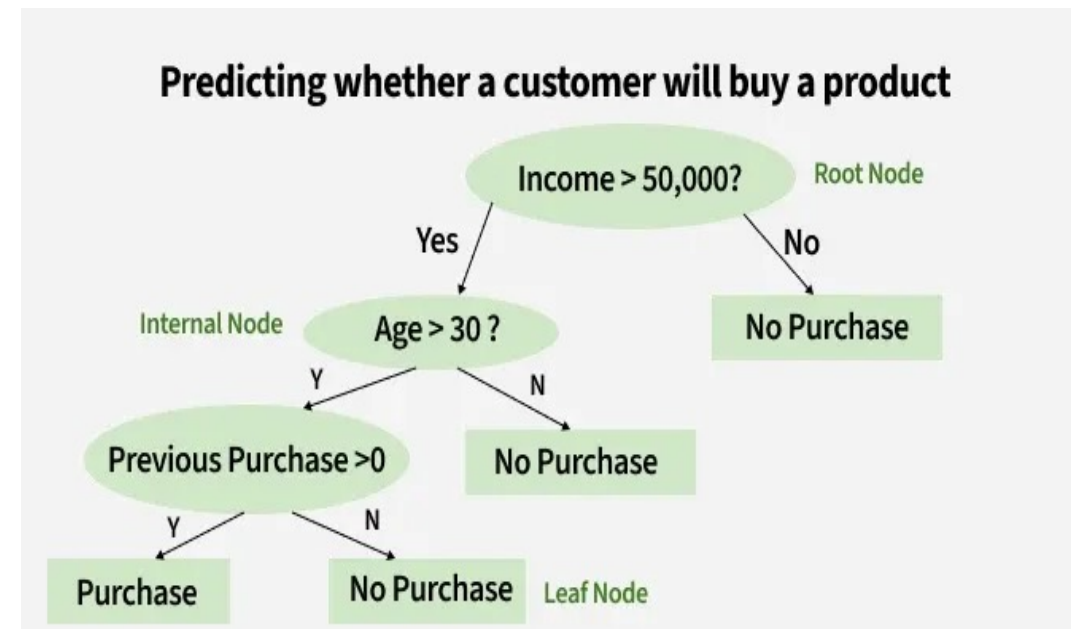
Next question: *"Is the age above 30?"*

1. Yes, move to the next step
2. No, predict "No Purchase"

3. Internal Node (Previous Purchases):

Final question: *"Has the person made previous purchases?"*

1. Yes, predict "Purchase"
2. No, predict "No Purchase"



Decision Tree types:

Classification Trees:

- It determines a set of logical if-then condition to classify problem.
- It is used when the target variable is categorical (such as Yes/No).

Regression Trees:

- It is used when the target variable is a numerical on continues in nature (such as predicting price or salary).
- It fits a regression model to target variable using each of independent variables.
- Each split is made based on the summation of the squared error.

Advantages of decision tree

It's simple to understand and interpret .

Little effort required for data preparation. No need for normalization or scaling of features.

Can handle both numerical and categorical data.

Nonlinear parameters don't affect its performance.

Captures nonlinear relationships. Can model complex patterns in data.

Entropy

- Entropy is the measure of randomness or unpredictability in the dataset.
- It characterizes the impurity of an arbitrary collection of examples.
- The higher the entropy more the information content.

$$\text{Entropy} = - \sum_i^n p(\text{values}_i) \cdot \log_2(p(\text{values}_i))$$

Entropy range is $(0, \log_2 n)$

Where **n** is the number of classes

Information Gain

- Information Gain: The information gain measures the decrease in entropy after the data set is split.
- It is calculated as follows:

$$IG(Y, X) = \text{Entropy}(Y) - \text{Entropy}(Y | X)$$

Entropy(Y):

Measures the uncertainty in the target variable Y (before splitting). How mixed or random the data is.

Entropy(Y | X):

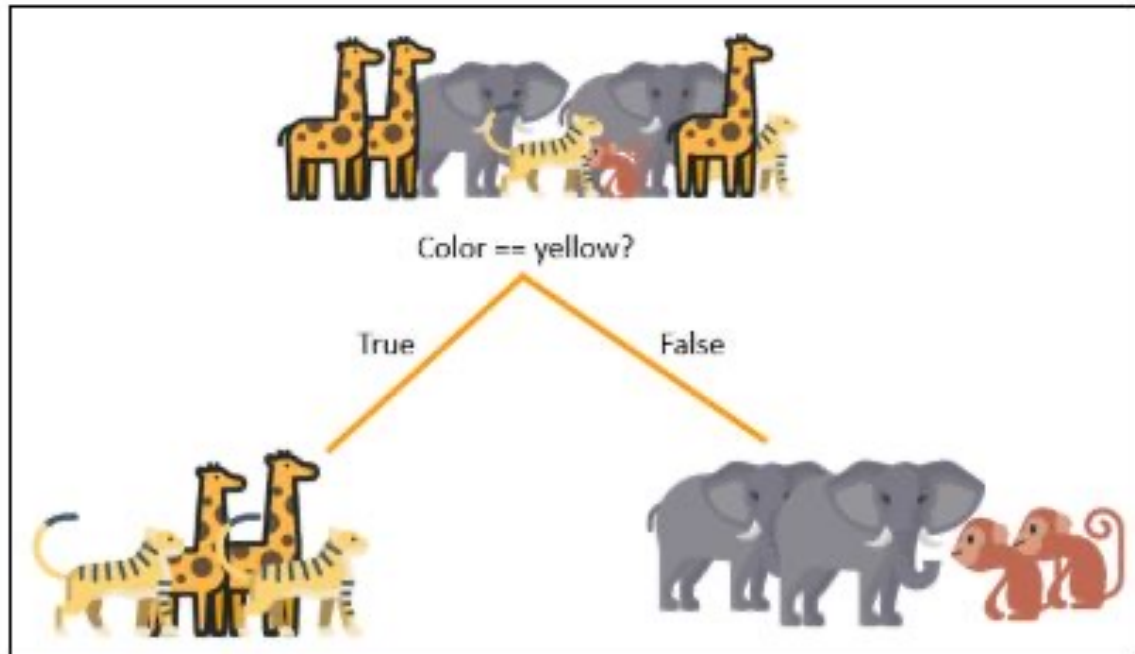
Measures the uncertainty in Y after splitting the data using feature X.

How mixed the groups are after the split.

Information Gain (IG):

The reduction in uncertainty after using feature X.

Example



Training Dataset

Color	Height	Label
Grey	10	Elephant
Yellow	10	Giraffe
Brown	3	Monkey
Grey	10	Elephant
Yellow	4	Tiger

Conditions

Color == Yellow

Height ≥ 10

Color == Brown

Color == Grey

Diameter < 10

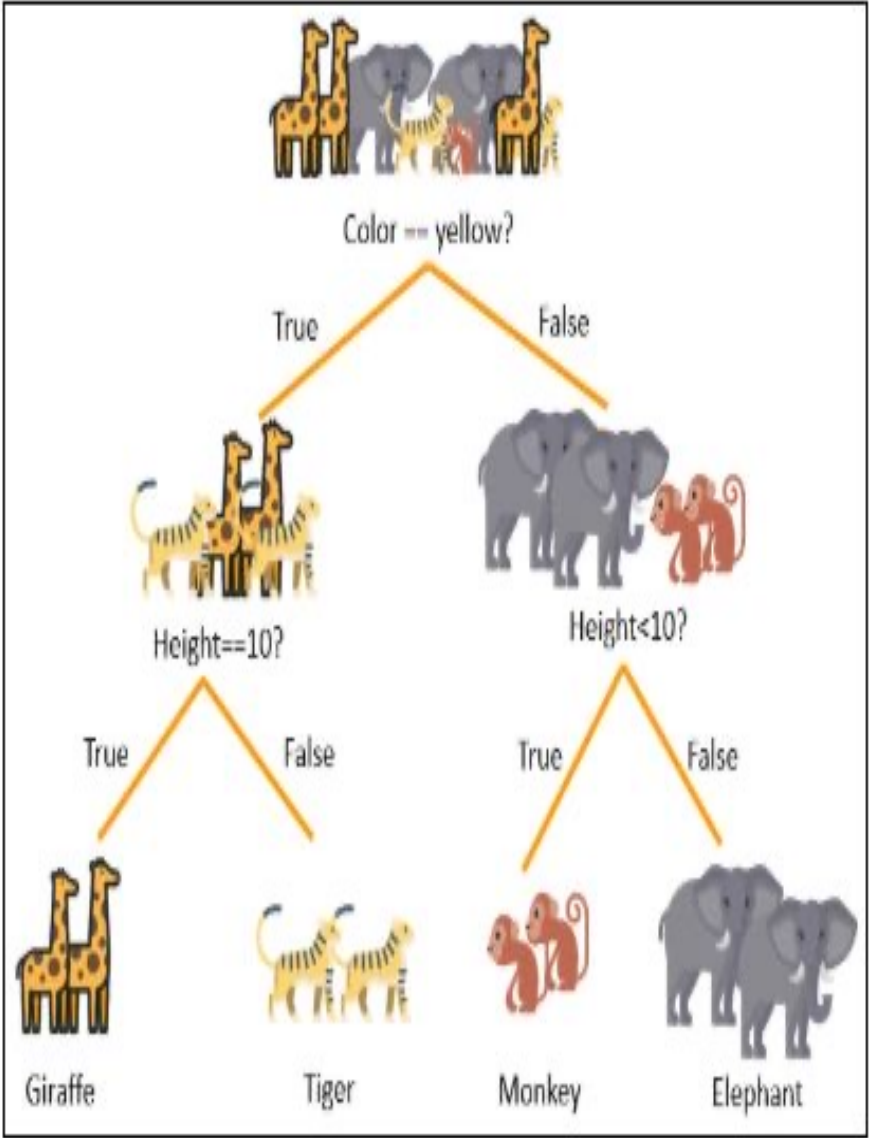
Entropy range is $(0, \log_2 n)$
 $= (0, \log_2 4) = (0, 2)$

Entropy = $-\sum_i^n p(\text{values}_i) \cdot \log_2(p(\text{values}_i))$
 $E1 = -[(\frac{2}{8} * \log_2(\frac{2}{8})) + (\frac{2}{8} * \log_2(\frac{2}{8})) + (\frac{2}{8} * \log_2(\frac{2}{8})) + (\frac{2}{8} * \log_2(\frac{2}{8}))]$
 $= -[0.25 * -2 + 0.25 * -2 + 0.25 * -2 + 0.25 * -2] = 2$

$E2_{\text{left}} = -[(\frac{2}{4} * \log_2(\frac{2}{4})) + (\frac{2}{4} * \log_2(\frac{2}{4}))] = 1$
 $E2_{\text{right}} = -[(\frac{2}{4} * \log_2(\frac{2}{4})) + (\frac{2}{4} * \log_2(\frac{2}{4}))] = 1$

Entropy(Y|X) or E2 = $\frac{4}{8} * (1) + \frac{4}{8} * (1)$
 $= 0.5 + 0.5 = 1$

Gain = $2 - 1 = 1$



High entropy E1

Low entropy E2

Gain = $E1 - E2$

Example:

Suppose we have 10 people, 6 **bought** the product (Yes) and 4 **did not buy** (No)

1. Calculate Entropy(Y) (before split)

$$\text{Entropy} = -\sum_i^n p(\text{values}_i) \cdot \log_2(p(\text{values}_i))$$

($p(\text{Yes}) = 6/10 = 0.6$) and ($p(\text{No}) = 4/10 = 0.4$)

$$\text{Entropy}(Y) = -(0.6 * \log_2 0.6 + 0.4 * \log_2 0.4) = 0.971$$

2. Split by Feature (Age: Young / Old)

- Young (5 people): 5 Yes, 0 No → pure, Entropy = 0
- Old (5 people): 1 Yes, 4 No

$$\text{Entropy}(\text{Old}) = -(0.2 * \log_2 0.2 + 0.8 * \log_2 0.8) = 0.722$$

3. Calculate Entropy(Y | X) average (after split)

$$\text{Entropy}(Y|X) = \frac{5}{10} * (0) + \frac{5}{10} * (0.722) = 0.361$$

$$\text{Entropy}(Y|X) = 0.361$$

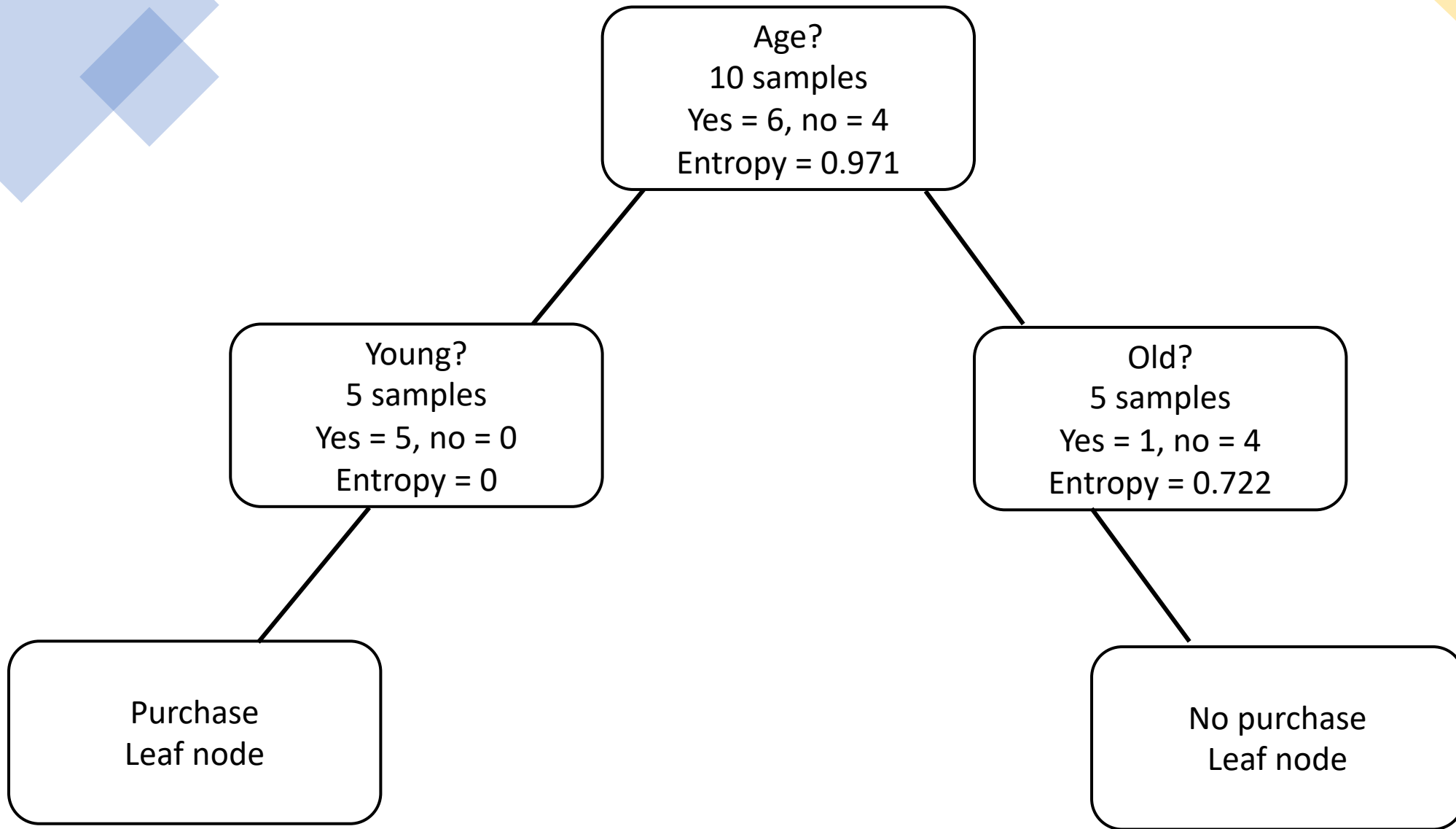
4. Calculate Information Gain

$$\text{IG}(Y, X) = \text{Entropy}(Y) - \text{Entropy}(Y|X)$$

$$\text{IG} = 0.971 - 0.361 = 0.61$$

- Entropy before split: 0.971 (high uncertainty)
- Entropy after split: 0.361 (lower uncertainty)
- Information Gain: 0.61
- Meaning: The split using Age significantly reduces uncertainty, so it is a good feature.

Note: big drop in entropy = high Information Gain = good split



Decision Tree algorithm in Python

- Decision Tree algorithm in Python to predict whether a customer is likely to repay a loan.
- The dataset includes several factors for this prediction, such as the initial payment, last payment, credit score, house number, and whether the loan was successfully repaid

Result	Initial Payment	Last Payment	Credit Score	House Number
yes	201	10018	250	3046
yes	205	10016	395	3044
yes	257	10129	109	3251
yes	246	10064	324	3137
yes	117	10115	496	3094

...

Code

```
#Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier

#Read csv file
df = pd.read_csv("/content/drive/MyDrive/AI course/dataset_DT.csv")
df.head()

#Trying to understand the dataset
df.info()
print(len(df))
df.isna().sum()
df.nunique()
```



```
#Dividing the dataset to independent variables and dependent variable
x = df.values[:,1:]
y = df.values[:,0]
print(y)

#Split the data to train and test parts
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,
random_state=100)

#Built the decision tree model
dtc = DecisionTreeClassifier(criterion="entropy",random_state=100,
max_depth=3,min_samples_leaf=5)
dtc.fit(x_train,y_train)

#Model evaluation
y_predict = dtc.predict(x_test)
accuracy_score(y_test,y_predict)
```

Decision Regression Trees

A Decision Tree Regressor is a non-parametric supervised learning method used to approximate a function.

$$F(\mathbb{R}^X) \rightarrow \mathbb{R}$$

- $\mathbb{R}^X$: Input features
- $\mathbb{R}$: Output is a single real number

It works by recursively dividing the feature space into non-overlapping regions and assigning a constant prediction to each region.

Model structure

Input space is divided into regions:

$$R_1, R_2, \dots, R_M$$

The model predicts:

$$\hat{y}(x) = \sum_{m=1}^m c_m \cdot 1(x \in R_m)$$

$$\hat{y}(x) = c_1 \cdot 0 + c_2 \cdot 0 + \dots + c_k \cdot 1 + \dots + c_M \cdot 0$$

Where:

- R_m = region (leaf node)
- M = total number of region
- c_m = average of all y values in region R_m
- 1 = indicator function

$$c_m = \frac{1}{|R_m|} \sum_{x \in R_m} y_i$$

- $|R_m|$: number of samples in region R_m
- Each leaf node predicts the average (mean) of the target values of the training samples that fall into that region.

Where:

$$x \in R_m = \begin{cases} 1 & \text{if } x \text{ is in region } R_m \\ 0 & \text{otherwise} \end{cases}$$

Splitting criterion

At each node, the algorithm searches for the best split:

- (j,s) Where:
- j = feature index
- s = split threshold
- The split divides data into:

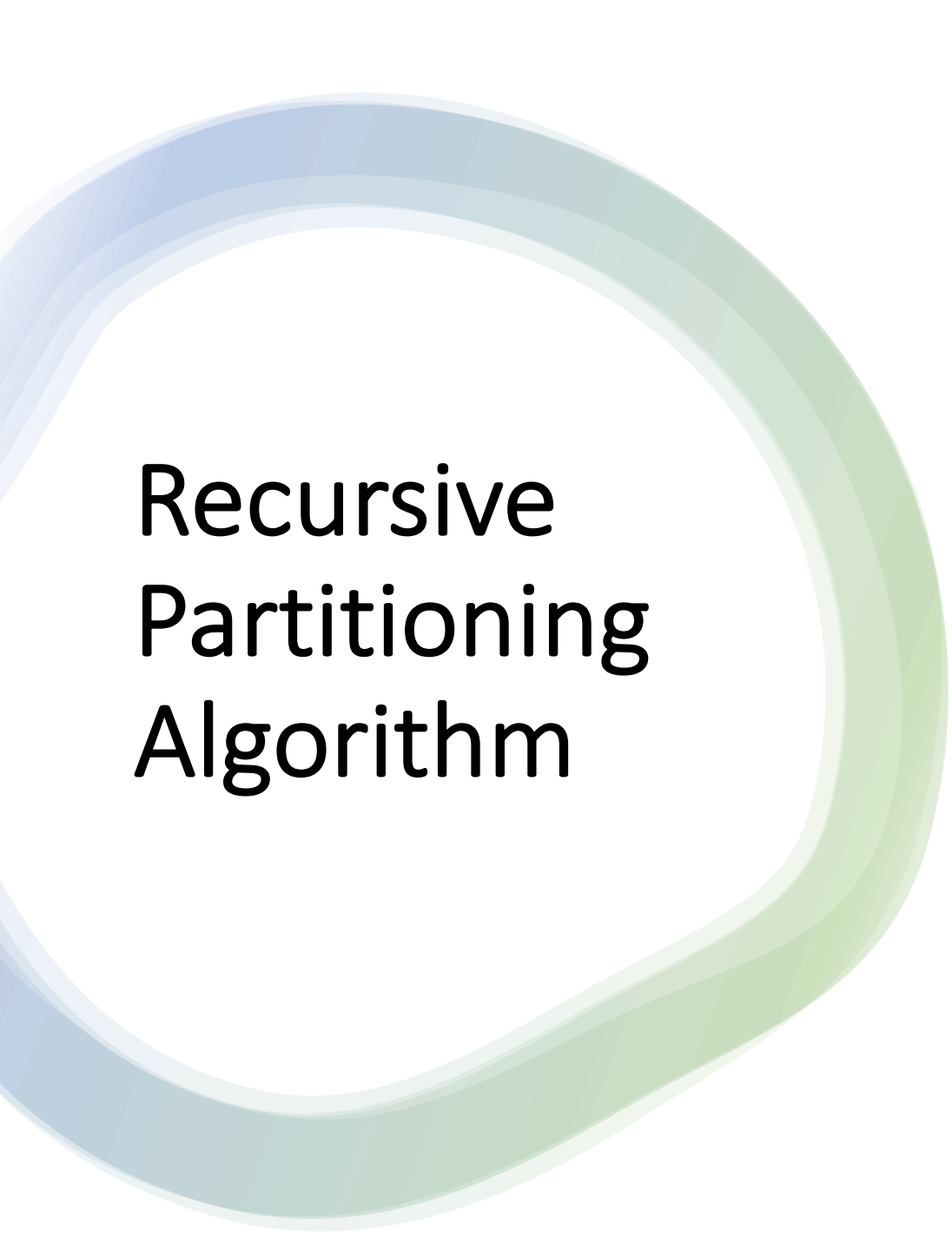
$$R_{\text{left}}(j,s)=\{x|x_j<s\}, R_{\text{right}}(j,s)=\{x|x_j\geq s\}$$

Objective function (training)

- The goal is to Minimize the sum of squared errors (SSE):

$$\text{Min}_{s,j} \left[\sum_{x_i \in R_{left}} (y_i - \hat{y}_{left})^2 + \sum_{x_i \in R_{right}} (y_i - \hat{y}_{right})^2 \right]$$

- This is equivalent to minimizing variance within nodes
-



Recursive Partitioning Algorithm

1. Start with all data (root node)
2. For each feature j and split s :
 Compute error SSE
3. Choose split with minimum error
4. Recursively repeat on each child node
5. Stop when:
 - Maximum depth reached
 - Minimum samples per leaf reached
 - No significant error reduction

Example:

Step1

First split: $x \leq 7$ vs $x > 7$

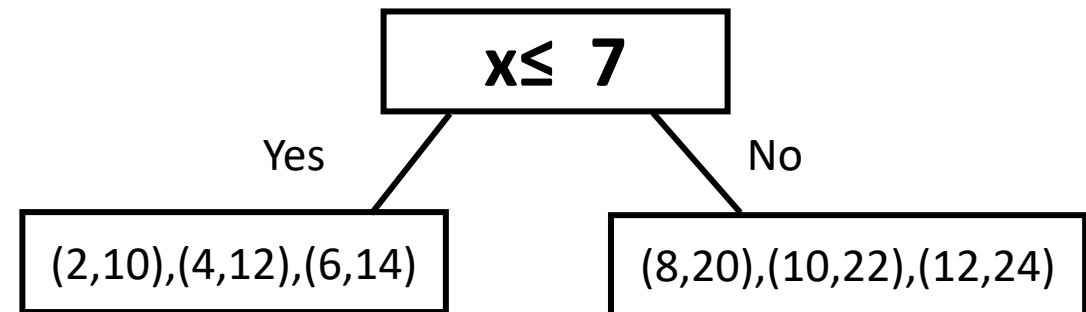
1. Region: $R1 = x \leq 7$

- Data points: (2,10),(4,12),(6,14)
- Compute mean: $c1 = \frac{10+12+14}{3} = 12$
- $\hat{y}(x) = 12$

2. Region: $R2 = x > 7$

- Data points: (8,20),(10,22),(12,24)
- Compute mean: $c2 = \frac{20+22+24}{3} = 22$
- $\hat{y}(x) = 22$

i	x	y
1	2	10
2	4	12
3	6	14
4	8	20
5	10	22
6	12	24



Step2:

Second split (recursive): $x \leq 9$ vs $x > 9$

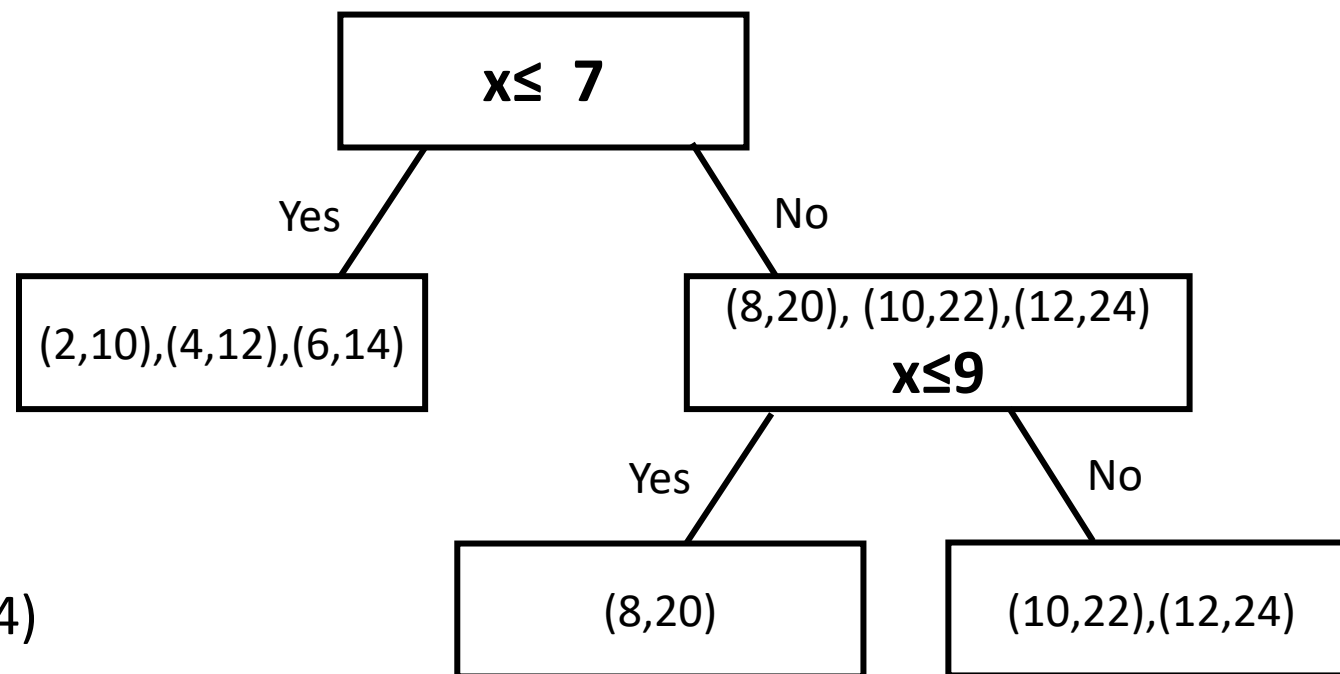
Region R2 becomes two regions:

1. Region: R2.1= $x \leq 9$

- Data points: (8,20)
- Compute mean: $c_{2.1}=20$
- $\hat{y}(x)=20$

2. Region: R2.2= $x > 9$

- Data points: (8,20),(10,22),(12,24)
- Compute mean: $c_{2.2} = \frac{22+24}{2} = 23$
- $\hat{y}(x)=23$

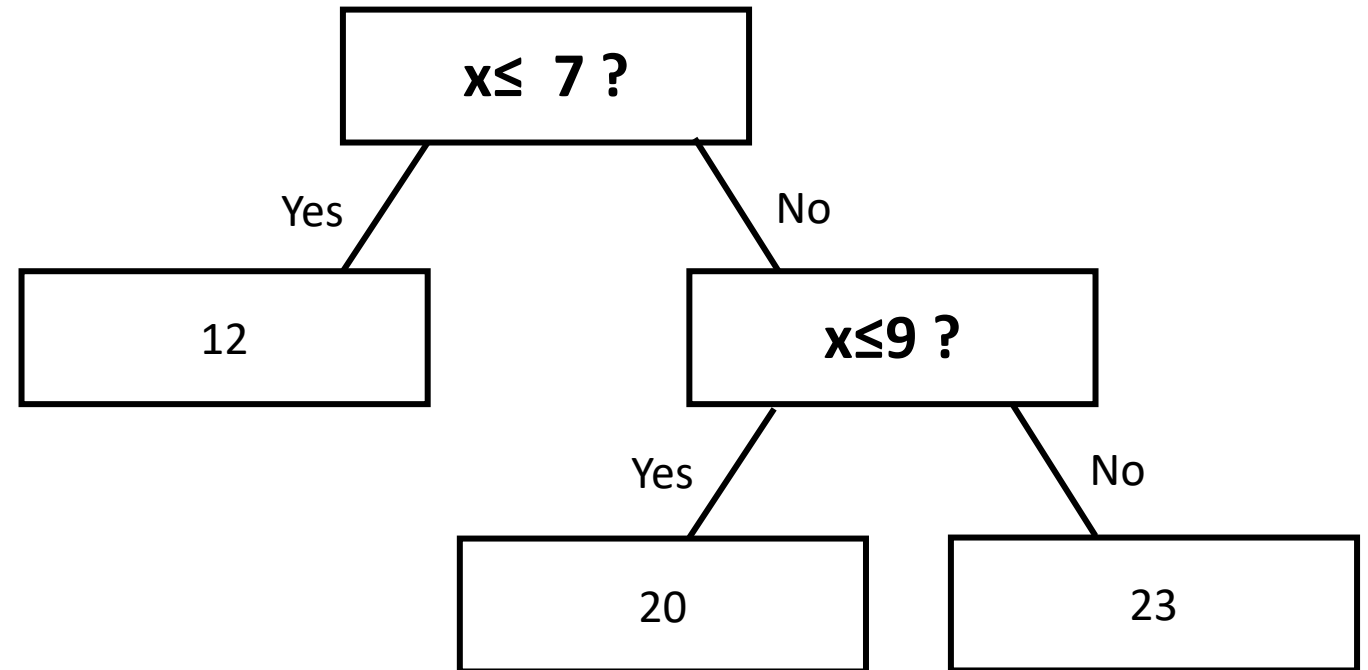


Final model:

Now we have:

- $R_1 = \{x \leq 7\}$, $c_1 = 12$
- $R_2 = \{7 < x \leq 9\}$, $c_2 = 20$
- $R_3 = \{x > 9\}$, $c_3 = 23$

x	Region	Prediction
3	R_1	12
6	R_1	12
8	R_2	20
11	R_3	23



Overfitting

Decision trees tend to overfit by growing too deep.

- **Prepruning, early stopping**

- max_depth

- min_samples_leaf

- min_samples_split

- **Post-pruning**

- Minimize:

$$\text{Cost} = \text{SSE} + \alpha \cdot |T|$$

Where:

- $|T|$ = number of leaves
 - α = learning ratio, only keep splits if they reduce SSE by more than α value.
 - $\alpha \rightarrow \text{ccp_alpha} = [0.0, 0.001, 0.01, 0.1]$

Decision Tree Hyperparameters		
max_depth	min_samples_split	min_samples_leaf
Small data: 2–5	Default: 2	Small data: 1–5
Medium: 3–10	recommended: 10–20	Medium: 5–20
Large: 5–20 or more		Large: more than 10

The goal is to balance overfitting and underfitting

Example:

Implement a Decision Tree model to predict the laptop price:

Company	TypeName	Ram	OpSys	Weight	Touchscreen	CpuCompany	ClockSpeed	Flash Storage	HDD	Hybrid	SSD	GPU	Ppi	Price
Apple	Ultrabook	8	macOS	1.37	0	Intel	2.3	0.0	0.0	0.0	0.128	Intel	312.78195488721800	71378.6832
Apple	Ultrabook	8	macOS	1.34	0	Intel	1.8	0.128	0.0	0.0	0.0	Intel	175.93984962406000	47895.5232
HP	Notebook	8	No	1.86	0	Intel	2.5	0.0	0.0	0.0	0.256	Intel	192.30769230769200	30636.0
Apple	Ultrabook	16	macOS	1.83	0	Intel	2.7	0.0	0.0	0.0	0.512	AMD	303.8961038961040	135195.33600000000
Apple	Ultrabook	8	macOS	1.37	0	Intel	3.1	0.0	0.0	0.0	0.256	Intel	312.78195488721800	96095.808
Acer	Notebook	4	Windows	2.1	0	AMD	3.0	0.0	0.5	0.0	0.0	AMD	136.7948717948720	21312.0
Apple	Ultrabook	16	Mac	2.04	0	Intel	2.2	0.256	0.0	0.0	0.0	Intel	303.8961038961040	114017.6016
Apple	Ultrabook	8	macOS	1.34	0	Intel	1.8	0.256	0.0	0.0	0.0	Intel	175.93984962406000	61735.536

```
#Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#Read csv file
df= pd.read_csv("/content/drive/MyDrive/AI course/laptop.csv")
df.head()
```

```
#Data Analysis in order to understand the dataset
```

```
df.info()
```

```
df.shape
```

```
df.isna().sum()
```

```
df.nunique()
```

```
#Dividing the dataset to independent variables and dependent variable
```

```
x = df.drop("Price",axis = 1)
```

```
x=pd.get_dummies(x,drop_first=True)
```

```
y = df["Price"]
```

```
x.head()
```

```
y.head()
```

```
#Split the data to train and test parts
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.23,  
random_state=2)
```

```
#Create Decision Tree Regression model
```

```
from sklearn.tree import DecisionTreeRegressor
```

```
DTR = DecisionTreeRegressor(max_depth= 18, min_samples_leaf=10,min_samples_split=5,ccp_alpha=0.1)
```

```
DTR.fit(x_train,y_train)
```

```
#Model evaluation(test part)
y_pred= DTR.predict(x_test)
from sklearn.metrics import r2_score
print("R2_test: ",r2_score(y_test,y_pred))
```

```
#Model evaluation(train part)
y_pred_train= DTR.predict(x_train)
print("R2_train: ",r2_score(y_train,y_pred_train))
```

```
#Draw the tree
plt.figure(figsize=(60,20))
from sklearn.tree import plot_tree
plot_tree(DTR,filled=True,feature_names=x.columns)
plt.show()
```